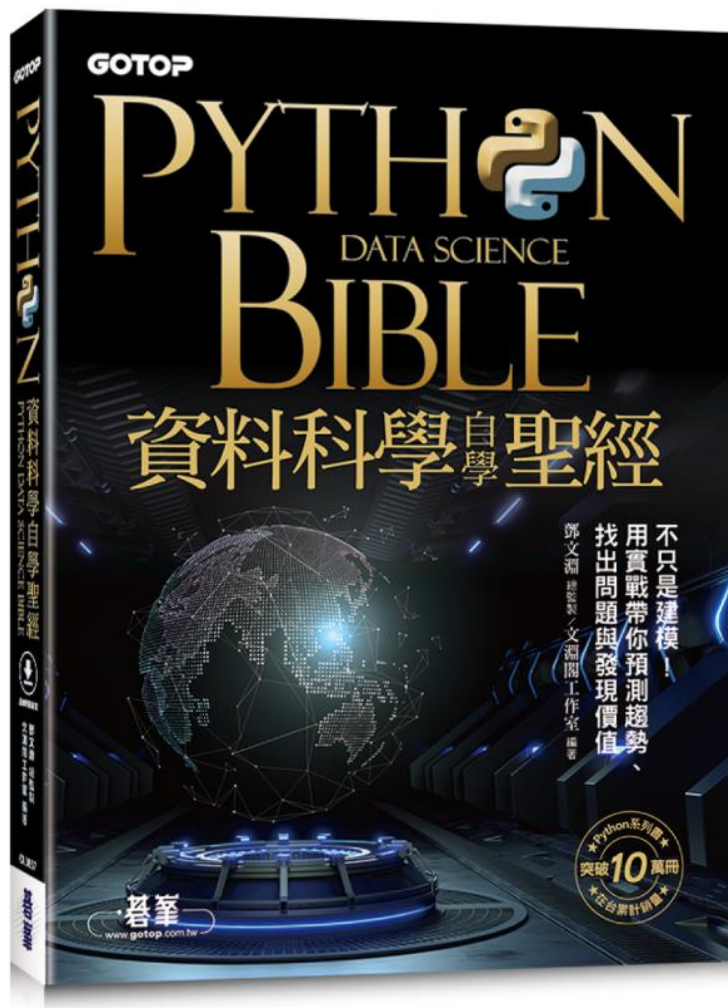




# PYTHON DATA SCIENCE BIBLE 資料科學自學聖經

碁峯資訊

【投影片使用規範與聲明】本投影片僅供非營利教學用途，教師得搭配用書授課講解使用，可於用書期間內將投影片置放於學校內部網站，但需有帳密權限機制，且僅供修課學生瀏覽使用。敬請老師善盡著作權保護之責，請勿將投影片任意散布與販售，亦不得以任何形式或方法轉載內容使用。

## 資料預處理： 資料清洗及圖片增量

[5.1 資料清洗處理](#)

[5.2 資料檢查](#)

[5.3 資料合併](#)

[5.4 樞紐分析表](#)

[5.5 圖片增量](#)

## 5.1 資料清洗處理

### 5.1.1 資料處理的相關工作

資料在收集時常常會包含一些不合乎要求的內容，例如資料欄位沒有值、資料重複、或是資料的值明顯不合常理等，這些內容都必須先進行處理，否則運用這些資料將會產生不正確的結果。

Pandas 提供了許多資料觀察以及相關處理的函式，常用的函式如下：

| 函式                             | 功能說明                  |
|--------------------------------|-----------------------|
| <code>info()</code>            | 顯示 DataFrame 所包含的內容資訊 |
| <code>describe()</code>        | 顯示 DataFrame 資料相關統計數據 |
| <code>isnull()</code>          | 篩選資料欄位為空值的記錄          |
| <code>dropna()</code>          | 刪除資料欄位為空值的記錄          |
| <code>fillna()</code>          | 填值到空值欄位               |
| <code>duplicated()</code>      | 檢測 DataFrame 中重複的記錄   |
| <code>drop_duplicates()</code> | 刪除重複的記錄               |

在進行資料處理之前，應該要先觀察資料的內容。建議先使用 DataFrame 的 `info()` 以及 `describe()` 函式，除了查看資料集的內容，還能以數據欄位的統計值對資料的內容進行了解。

## 取得資料集的資料摘要：df.info()

使用df.info() 函式用於獲取 DataFrame 的簡要摘要，包括索引及每個欄位的資料型態，非空值(non-null) 和記憶體的使用情況，在對數據資料進行探索性分析時會非常方便。

```
[3] 1 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 30 entries, 0 to 29  
Data columns (total 9 columns):  
#   Column  Non-Null Count  Dtype    
--  --  --  --  --  --  
0   學號      30 non-null    object   
1   姓名      30 non-null    object   
2   性別      30 non-null    object   
3   電郵      30 non-null    object   
4   國文      28 non-null    float64  
5   英文      30 non-null    int64     
6   數學      29 non-null    float64  
7   歷史      29 non-null    float64  
8   地理      29 non-null    float64  
dtypes: float64(4), int64(1), object(4)  
memory usage: 2.2+ KB
```

資料筆數及索引值範圍  
資料欄位數  
資料欄位資訊名稱  
資料欄位內容  
記憶體使用

## 取得資料集的資料摘要：df.describe()

使用df.describe() 函式用於產生資料的描述性統計數據，用來觀察數據集分佈的集中趨勢，分散和形狀，資料之中不包括空值。例如，請使用剛才載入資料的DataFrame，使用describe() 函式顯示摘要資料：

```
[4] 1 df.describe()
```

|       | 國文        | 英文        | 數學        | 歷史        | 地理        |
|-------|-----------|-----------|-----------|-----------|-----------|
| count | 28.000000 | 30.000000 | 29.000000 | 29.000000 | 29.000000 |
| mean  | 70.071429 | 76.766667 | 75.793103 | 76.793103 | 76.068966 |
| std   | 24.452416 | 19.162973 | 21.645487 | 22.266854 | 25.516005 |
| min   | -1.000000 | -1.000000 | -1.000000 | -1.000000 | -1.000000 |
| 25%   | 64.750000 | 68.000000 | 63.000000 | 62.000000 | 70.000000 |
| 50%   | 77.500000 | 84.000000 | 83.000000 | 82.000000 | 83.000000 |
| 75%   | 86.000000 | 88.000000 | 90.000000 | 93.000000 | 91.000000 |
| max   | 94.000000 | 97.000000 | 99.000000 | 99.000000 | 99.000000 |

## 5.1.2 缺失值處理

通常資料清洗的第一步是檢查資料是否含有缺失值，因為若資料含有缺失值，常會造成後續資料處理程序產生錯誤。

### 顯示有空值的資料：`df.isnull()`

如果想要顯示欄位有空值的資料記錄，可以使用`df.isnull()` 函式，語法為：

```
條件變數 = DataFrame 變數 .isnull(). 空值型態 (axis='columns')  
DataFrame 變數 [ 條件變數 ]
```

- **空值型態**：空值型態可能值有兩種：「all」表示資料所有欄位都是空值才符合條件，「any」表示資料任何欄位有空值就符合條件。

### 移除有空值的資料：`df.dropna()`

有空值的資料該如何處理呢？根據資料狀況，通常有兩種方式：**移除空值資料**或**填補空值資料**。

**處理空值資料最簡單的方式是將含有空值的資料刪除**。這種方式適用於資料數量龐大的情況，因為資料很多，刪除少量含有空值的資料不會影響資料分析結果。

刪除空值資料的語法為：

**DataFrame 變數 .dropna(how= 空值型態 , thresh= 數值 , subset= 欄位串列 )**

- **how**：非必填，可能值有兩種：「all」表示資料所有欄位都是空值才刪除資料，「any」為預設值，表示資料任何欄位有空值就刪除資料。
- **thresh**：非必填，表示非空值欄位小於參數值就刪除資料。
- **subset**：非必填，參數值是欄位名稱組成的串列，表示在串列中的欄位若有空值就刪除資料。

## 空值資料填補:df.fillna()

如果資料數量不多，希望所有資料都能保留，就要給予空值資料適當的值，避免資料分析時產生錯誤。

空值資料填補的語法為：

**DataFrame 變數 .fillna(value= 數值 , method= 填充位置 , axis= 列或行 )**

- **value**：此參數可以是固定數值，也可以使用DataFrame 的統計函式來取得計算結果。例如平均值mean()、中位數median()、最大值max()、最小值min()等。例如考試成績常會填補平均值，如此資料分析時就不會影響平均成績。設定此參數時，「value=」可以省略。
- **method**：此參數可以設定以鄰近資料來填補。參數值為「backfill」或「bfill」表示以下一個資料填補，參數值為「ffill」或「pad」表示以上一個資料填補。
- **axis**：非必填，若有設定 method 參數則可以此參數設定填補是列或行。參數值為「index」或「0」表示以列資料填補，此為預設值；參數值為「columns」或「1」表示以行資料填補。
- **注意**：value 及method 參數只能設定一個，若兩個都設定，執行會產生錯誤。



## 5.1.3 重複資料處理

有時資料中會有完全相同的資料，可以使用`df.duplicated()` 函式找出來，例如：

```
[47] 1 df = pd.read_csv('學生月考成績檔.csv')
      2 df.duplicated()

0    False
1    False
2    False
3    False
4    False
5    False
6    False
7     True
8    False
```

可以使用`df.drop_duplicates()` 函式刪除重複資料，語法為：

**DataFrame 變數 .drop\_duplicates([keep= 刪除型態, ignore\_index= 布林值])**

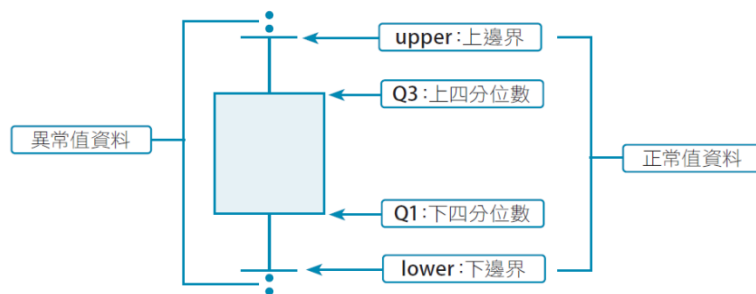
- **keep**：非必填，是設定要保留哪一筆資料。可能值有三種：「first」表示保留第一個重複資料，其餘刪除，「last」表示保留最後一個重複資料，其餘刪除，「False」表示不保留，刪除全部重複資料。預設值為first。
- **ignore\_index**：非必填，是設定刪除後是否重新建立索引值：True 表示重新建立索引，False 表示不會重新建立索引。預設值為False。



## 5.1.4 異常值處理

**異常值** 是指數值資料中含有某些特別大或特別小的值，**注意：只有數值型欄位資料才存在異常值的問題。**

尋找異常值資料的方法很多，較常用的方法是「箱形圖」法。以下以12 個資料「32,40,41,41,39,40,40,42,50,40,40,41,41,42,58」做為範例說明「線箱圖」法的步驟：



1. 將資料由小到大排序：「32,39,40,40,40,40,41,41,42,42,50,58」。
2. 計算Q1(下四分位數)：前6 個資料中位數 $= (40+40)/2=40$ 。
3. 計算Q3(上四分位數)：後6 個資料中位數 $= (42+42)/2=42$ 。
4. 計算IQR (四分位間距)： $Q3 - Q1 = 2$ 。
5. 計算lower (下邊界)： $Q1 - 1.5 * IQR = 40 - 1.5 * 2 = 37$ 。
6. 計算upper (上邊界)： $Q3 + 1.5 * IQR = 42 + 1.5 * 2 = 45$ 。
7. 大於上邊界或小於下邊界者為異常：「32、50、58」為異常值。

## 5.2 資料檢查

### 5.2.1 範圍檢查

範圍檢查是指檢查資料數值是否在合理的範圍內，例如檢查學生成績是否在0 分到 100 分之間，又如檢查攝氏體溫記錄是否在34 度到45 度之間。

以下程式會檢查所有成績數值，若成績為負值就將其更改為0 分：

```
[63] 1 df.loc[df['國文'] < 0, '國文'] = 0  
      2 df.loc[df['英文'] < 0, '英文'] = 0  
      3 df.loc[df['數學'] < 0, '數學'] = 0  
      4 df.loc[df['歷史'] < 0, '歷史'] = 0  
      5 df.loc[df['地理'] < 0, '地理'] = 0  
      6 df
```

### 5.2.2 資料格式檢查

通常資料格式檢查會使用 **正規表達式**(Regular Expression，簡稱regex) 來進行。正規表達式簡單來說就是用一定的規則處理字串的方法。它能透過一些特殊符號的輔助，讓使用者輕易達到「搜尋/ 取代」資料中某些特定字串的處理。關於正規表達式的使用方式，請自行參考相關資料。

在字串中尋找指定格式子字串的語法為：

字串變數 `.contains(pat[, case= 布林值 , na= 填補值 , regex= 布林值 ])`

- **pat**：要尋找的子字串或正規表達式。若 **regex** 參數值為 **False**，則此參數值為要尋找的子字串；若 **regex** 參數值為 **True**，則此參數值為正規表達式。
- **case**：非必填，預設值為 **True**，表示尋找的子字串要區分大小寫，**False** 則否。
- **na**：非必填，表示若為缺失值時替換為此參數值。
- **regex**：非必填，預設值為 **True**，表示 **pat** 參數值為正規表達式，**False** 表示 **pat** 參數值為字串。

下面是檢查常用格式的正規表達式：

| 格式             | 正規表達式                                                                                                                               |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------|
| 電子郵件           | <code>^([a-zA-Z0-9._%~]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,4})*\$</code>                                                                   |
| URL 網址         | <code>^(?:(https? ftp):\/\/)?((?:[a-zA-Z0-9.\-]+\.)+(?:[a-zA-Z0-9]{2,4}))((?:/[w+=%&amp;~\-\*]*)*)\??(?:[w+=%&amp;~\-\*]*)\$</code> |
| IP 位址          | <code>^((?:25[0-5] 2[0-4][0-9] [01]?[0-9][0-9]?)\.){3}(?:25[0-5] 2[0-4][0-9] [01]?[0-9][0-9]?)\$</code>                             |
| 身份證字號          | <code>^[A-Za-z][1-2]\d{8}\$</code>                                                                                                  |
| 手機號碼           | <code>^09\d{2}-?\d{3}-?\d{3}\$</code>                                                                                               |
| Visa 信用卡       | <code>^(4[0-9]{12}(?:[0-9]{3})?)*\$</code>                                                                                          |
| MasterCard 信用卡 | <code>^(5[1-5][0-9]{14})*\$</code>                                                                                                  |

以下程式可篩選出電子郵件欄位格式不符的資料：

```
[70] 1 import pandas as pd
      2 df = pd.read_csv('學生月考成績檔.csv')
      3 column = '電郵'
      4 condition = df[column].str.contains(
      5     r'^([a-zA-Z0-9._%~]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,4})*$',
      6     regex=True)
      7 df[~condition]
```

## 5.3 資料合併

資料收集的過程中常會將不同的資料分布在不同檔案中，可以使用Pandas 讀取後再將資料合併。

### 5.3.1 資料附加

資料附加是最簡單的資料合併方法，功能是將資料加在原始資料後方。資料附加的語法為：

```
DataFrame 變數 1.append(DataFrame 變數 2, ignore_index=布林值)
```

- **ignore\_index**：非必填，設定資料附加後是否重新建立索引值：True 表示重新建立索引，False 是預設值，表示不會重新建立索引。

資料附加時若有欄位不相符時，會保留所有欄位，原始資料中沒有的欄位資料會以空值填充。

例如，<score\_1.csv> 有國文、數學、自然三科，<score\_3.csv> 有國文、數學、社會三科，則附加後自然及社會科都會保留，原來沒有的資料會以空值填充。

```
[4] 1 import pandas as pd
    2 df1 = pd.read_csv('score_1.csv')
    3 df2 = pd.read_csv('score_3.csv')
    4 df1.append(df2, ignore_index=True)
```

|   | 座號 | 國文  | 數學 | 自然   | 社會   |  |
|---|----|-----|----|------|------|--|
| 0 | 1  | 58  | 40 | 45.0 | NaN  |  |
| 1 | 2  | 48  | 46 | 55.0 | NaN  |  |
| 2 | 3  | 81  | 89 | 74.0 | NaN  |  |
| 3 | 4  | 100 | 42 | NaN  | 95.0 |  |
| 4 | 5  | 78  | 90 | NaN  | 47.0 |  |
| 5 | 6  | 98  | 58 | NaN  | 67.0 |  |

原始資料中沒有的欄位資料會以空值填充

## 5.3.2 資料串接

資料串接的功能與資料附加雷同，也是將資料加在原始資料後方，只是多了參數可以指定欄位合併的方式。

資料串接的語法為：

```
pd.concat([DataFrame 變數 1, DataFrame 變數 2],
           ignore_index=布林值, join=合併方式)
```

- **ignore\_index**：非必填，是設定資料附加後是否重新建立索引值。
- **join**：非必填，設定值如下：
  - **outer**：意義為「聯集」，會保留所有欄位與append效果相同，此為預設值。
  - **inner**：意義為「交集」，只會保留共同欄位資料。

### 5.3.3 資料融合

資料融合是功能最強大的資料合併方式，也是使用最多的資料合併方式。資料融合的語法為：

```
pd.merge(DataFrame 變數 1, DataFrame 變數 2  
         , left_on= 欄位, right_on= 欄位, on= 欄位  
         , suffixes=[ 後綴 1, 後綴 2, …… ], how= 合併方式 )
```

- **left\_on**：非必填，設定第一個資料集合併欄位基準。
- **right\_on**：非必填，設定第二個資料集合併欄位基準。
- **on**：非必填，設定兩個資料集共同合併欄位基準。
- **suffixes**：非必填，設定值是一個串列。如果要合併的資料集有相同名稱的欄位，此設定值是為合併後的相同名稱欄位加上後綴文字做為區別。
- **how**：非必填，可能值有四種：
  - **outer**：意義為「聯集」，會保留所有欄位。
  - **inner**：意義為「交集」，只會保留共同欄位資料。此為預設值。
  - **left**：保留第一個資料集所有欄位資料。
  - **right**：保留第二個資料集所有欄位資料。



## 一對一融合

系統會自動以共同的「座號」欄位進行融合。預設是以「inner (交集)」融合，即共同的座號才保留。

## 多對一融合

例如，<score\_1.csv> 有一個座號1 的資料，<score\_5.csv> 有三個座號1 的資料，融合後會保留三個座號1 的資料，且複製<score\_1.csv> 座號1 的資料來補足。

## 指定融合參考欄位

在<score\_1.csv> 與<score\_6.csv> 這兩個資料集有座號及數學兩個共同欄位，此時必須以left\_on 及right\_on 參數指定融合參考欄位，系統才能進行融合。例如，以「座號」進行融合，融合後系統會自動為相同的欄位名稱後面加上「\_x」及「\_y」後綴做為區別。

## 自訂欄位名稱後綴文字

在剛才的範例中，Pandas 自動為相同欄位名稱加上的後綴常不符需求，可使用「suffixes」參數自行設定後綴文字。例如，將前面範例後綴改為「\_自」及「\_社」，表示自然組數學及社會組數學。

## 5.4 樞紐分析表

樞紐分析表 是具有交互功能的表格，可以將資料分成群組、以不同的方式來檢視。

### 5.4.1 樞紐分析表語法

樞紐分析表的語法為：

```
pd.pivot_table(DataFrame 變數, index= 列欄位, columns= 行欄位,
                values= 分析欄位, margins= 布林值, margins_name= 字串,
                aggfunc= 統計項目, fill_value= 值, dropna= 布林值)
```

- **index**：此參數為必要參數，是一個串列，功能是設定要分析的「列」欄位。若有多個列欄位，結果會以巢狀方式呈現。
- **columns**：非必填，是一個串列，功能是設定要分析的「行」欄位。若有多個行欄位，結果會以巢狀方式呈現。
- **values**：非必填，是一個串列，功能是設定要進行統計的欄位。
- **margins**：非必填，True 時表示要計算總和，False 時表示不要計算總和。預設值為False。

- **margins\_name**：非必填，當 margins 參數為 True 時才有效，功能是設定總和欄位的名稱。預設值為「All」。
- **aggfunc**：非必填，是一個串列，功能是設定要統計的項目。常用的統計項目有mean ( 平均)、sum ( 總和)、max ( 最大值)、min ( 最小值)、count ( 次數)。預設值為mean。
- **fill\_value**：非必填，功能是設定資料為空值時以此設定值填充。預設值為None。
- **dropna**：非必填，True 時表示要刪除空值資料，False 時表示不刪除空值資料。預設值為True。

## 5.4.2 樞紐分析表實作

例如，以下程式會以「業務員」分組統計資料。因為沒有設定values 參數，則除了「業務員」以外的欄位都會統計，預設統計項目為「平均值」，此處可看到每個業務員的各項統計資料。

```
[9] 1 import pandas as pd
    2 df1 = pd.read_csv('商品銷售表.csv')
    3 pd.pivot_table(df1, index=['業務員'])
```

| 業務員 | 價格           |            | 數量  |  |
|-----|--------------|------------|-----|--|
|     | 張三安          | 李四友        | 王五信 |  |
| 張三安 | 14890.196078 | 155.607843 |     |  |
| 李四友 | 15815.789474 | 171.473684 |     |  |
| 王五信 | 15000.000000 | 174.266667 |     |  |

**注意:**文字資料無法進行統計運算，因此樞紐分析表不會包含文字資料欄位。

## 5.5 圖片增量

圖片增量技術是從既有的圖片中產生出更多的圖片讓系統去學習，意即是創造更多的「假」圖片，來彌補圖片不足的缺憾。雖然是假的圖片，但也是從原始圖片內容修改產生的，而且此技術的確可解決圖片不足的困境，提昇系統訓練的準確率。

一張圖片經過旋轉、調整大小、比例尺寸，或者改變亮度、色彩、翻轉等處理後，人眼仍能辨識出來是相同的相片，但是對機器來說則是完全不同的新圖像了，因此，將既有的圖片予以修改變形，就能創造出更多的圖片來讓機器學習，彌補資料量不足的困擾。

## 5.5.1 keras ImageDataGenerator 模組

Colab 預設已安裝keras 模組，要使用圖片增量功能需先載入 ImageDataGenerator 模組，語法為：

```
from keras.preprocessing.image import ImageDataGenerator
```

接著建立ImageDataGenerator 物件，語法為：

```
增量變數 = ImageDataGenerator( 參數 1= 值 1, 參數 2= 值 2, ……)
```

- **featurewise\_center**：將輸入的特徵資料平均值設為 0。預設值為False。
- **samplewise\_center**：將每張圖片資料的平均值設為 0。預設值為False。
- **featurewise\_std\_normalization**：將輸入的資料除以標準差，依特徵逐一執行。預設值為False。
- **samplewise\_std\_normalization**：將輸入的資料除以標準差，依圖片逐一執行。預設值為False。
- **zca\_whitening**：zca 白化效果，可對圖片降維，減少圖片冗餘訊息。預設值為False。

- **rotation\_range**：設定圖片旋轉角度範圍。此設定並不是固定以這個角度進行旋轉，而是在0 到設定角度範圍內進行隨機角度旋轉。後面有關設定值皆是如此：取值為在設定值範圍內取值。
- **width\_shift\_range**：設定水平位置平移距離，其最大平移距離為圖片長度乘以此設定值。平移圖片的時候常會出現超出原圖範圍的區域，這部分區域會根據「fill\_mode」參數的設定來進行填補。
- **height\_shift\_range**：設定垂直位置平移距離，其最大平移距離為圖片高度乘以此設定值。
- **shear\_range**：設定剪切變形範圍，效果就是讓所有點的x坐標(或者y坐標)保持不變，而對應的y 坐標( 或者x 坐標) 則按比例發生平移。( 下圖虛線矩形為原始圖形，實線平行四邊形是剪切變形的範例)
- **zoom\_range**：設定圖片縮放範圍。參數值大於 0 小於 1 時，執行的是放大操作；當參數大於1 時，執行的是縮小操作。
- **channel\_shift\_range**：設定改變圖片顏色的程度。當數值越大時，顏色變深的效果越強。

- **horizontal\_flip**：設定是否會隨機水平翻轉。預設值為False。
- **vertical\_flip**：設定是否會隨機垂直翻轉。預設值為 False。
- **rescale**：設定對圖片的每個像素值均乘上這個參數值，這個操作在所有其他變換操作之前執行。
- **fill\_mode**：設定對圖片空白處的填補方式。可能值有 constant、nearest、reflect、wrap，預設值為nearest。經實測發現「reflect」效果最好。

接著利用ImageDataGenerator 物件的flow() 方法產生圖片，語法為：

產生變數 = 增量變數 .flow( 圖片路徑 , 參數 1= 值 1, |參數 2= 值 2, ……)

- **圖片路徑**：原始圖片檔案路徑。注意圖片格式必須是 numpy 陣列，且維度必須是4 維。
- **batch\_size**：每次處理的圖片數量。數值越大，處理速度越快，耗費記憶體越多。預設值為32。



- **save\_to\_dir**：儲存圖片路徑。若設定此參數會將產生的圖片存檔，若未設定此參數，產生的圖片不會存檔。
- **save\_prefix**：儲存圖片檔名前綴字串。當圖片存檔時程式會自動產生隨機檔名，這個參數可以為這些隨機檔名加上前綴字串，在管理上更容易識別。
- **save\_format**：儲存圖片格式，可能值為 png 或 jpg。預設值為 png。

取得產生圖片的語法為：

```
圖片變數 = 產生變數 [0][0, :, :, :].astype(np.uint8)
```

## 圖片自動產生

ImageDataGenerator 物件參數頗多，可以僅設定一個變形參數來觀看其產生的圖片效果。以下程式設定「`shear_range=50.0`」讓使用者觀看圖片剪切效果。



```
1 from keras.preprocessing.image import ImageDataGenerator
2 import matplotlib.pyplot as plt
3 import numpy as np
4 import cv2
5
6 imgGen = ImageDataGenerator(shear_range=50.0, fill_mode='reflect')
7 n = 5
8 img = cv2.imread('cat.jpg')
9 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) #BGR轉成RGB
10 img_sr = img.copy()
11 img = np.array(img, dtype=np.float32)
12 img = np.expand_dims(img, 0) #輸入圖片要是四維
13 flowGen = imgGen.flow(img, batch_size=10)
14
15 plt.figure(figsize=(20,10))
16 plt.subplot(1, n+1, 1)
17 plt.imshow(img_sr)
18 for i in range(n):
19     img = flowGen[0][0, :, :, :].astype(np.uint8)
20     plt.subplot(1, n+1, i+2)
21     plt.imshow(img)
22     plt.axis('off')
```

## 儲存自動產生的圖檔

如果要將產生的圖片存檔，可在flow() 方法設定save\_to\_dir、save\_prefix 及 save\_format 參數即可。例如，在剛才的範例中產生了圖片後儲存的方式如下：

```
6  imgGen = ImageDataGenerator(shear_range=50.0, fill_mode='reflect')
7  n = 5
8  img = cv2.imread('cat.jpg')
9  img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) #BGR轉成RGB
10 img = np.array(img, dtype=np.float32)
11 img = np.expand_dims(img, 0) #輸入圖片要是四維
12 flowGen = imgGen.flow(img, save_to_dir='.',
13                        save_prefix='cat',
14                        save_format='jpg',
15                        batch_size=10)
16
17 for i in range(n):
18     img = flowGen[0][0, :, :, :].astype(np.uint8)
```

## 5.5.2 augmentor 模組

### 模組安裝與基本語法

安裝augmentor 模組的語法為：

```
!pip install augmentor
```

使用augmentor 模組需先載入augmentor 模組：

```
import Augmentor
```

然後建立pipeline 物件，語法為：

```
pipeline 物件 = Augmentor.Pipeline( 資料夾路徑 )
```

- **資料夾路徑**：此參數為包含要產生圖片的原始圖片資料夾路徑，原始圖片可為 1 張或多張。

### 模組圖片的處理方法

接著加入圖片處理方法，語法為：

```
pipeline 物件.處理方法 ( 參數 1= 值 1, 參數 2= 值 2, ……)
```

常用的處理方法及參數有：

## 1. 翻轉 (flip\_left\_right、flip\_top\_bottom、flip\_random)：

```
flip_left_right(probability= 浮點數) # 水平翻轉  
flip_top_bottom(probability= 浮點數) # 垂直翻轉  
flip_random(probability= 浮點數)    # 水平或垂直翻轉
```

- **probability**：數值在 0 到 1 之間，表示產生處理動作的機率。例如設為 0.4，表示產生 10 張圖片時會有 4 張圖片進行翻轉。

## 2. 彈性扭曲(random\_distortion)：

```
random_distortion(probability= 浮點數, grid_width= 整數,  
                  grid_height= 整數, magnitude= 整數)
```

- **grid\_width**：水平扭曲距離，值為 1 到 20 的整數。
- **grid\_height**：垂直扭曲距離，值為 1 到 20 的整數。
- **magnitude**：扭曲強度，值為 1 到 20 的整數。

### 3. 放大縮小(zoom)：

```
zoom(probability= 浮點數, min_factor= 浮點數, max_factor= 浮點數)
```

- min\_factor：最小縮放比例。

- max\_factor：最大縮放比例。

### 4. 傾斜扭曲(skew)：

```
skew(probability= 浮點數, magnitude= 浮點數)
```

- magnitude：扭曲強度，為 0.1 到1.0 的浮點數。

### 5. 亮度(random\_brightness)：

```
random_brightness(probability= 浮點數, min_factor= 浮點數,  
max_factor= 浮點數)
```

- min\_factor：最小亮度因子。

- max\_factor：最大亮度因子。

### 6. 旋轉(rotate)：

```
rotate(probability= 浮點數, max_left_rotation= 整數,  
max_right_rotation= 整數)
```

- max\_left\_rotation：最大向左方旋轉角度。

- max\_right\_rotation：最大向右方旋轉角度。

### 7. 產生雜點(random\_erasing)：

```
random_erasing(probability= 浮點數, rectangle_area= 浮點數)
```

- rectangle\_area：雜點矩形面積，為 0.1 到1.0 的浮點數。

## 模組產生圖片的語法

最後以pipeline 物件的sample 即可產生圖片，語法為：

■ 數量：產生的圖片總數。

```
pipeline 物件.sample(數量)
```

## 範例：由單一圖片套用單一效果產生多張圖片

以下程式會以彈性扭曲效果產生4 張圖片。

```
1 import Augmentor
2 p = Augmentor.Pipeline("./image1")
3 p.random_distortion(probability=0.5,
4                       grid_height=10,
5                       grid_width=10,
6                       magnitude=10)
7 p.sample(4)
```

## 範例：由單一圖片套用多個效果產生多張圖片

請建立<image2> 資料夾後上傳<cat.jpg> 圖片檔，再執行以下程式：

```
1 import Augmentor
2 p = Augmentor.Pipeline("./image2")
3 p.flip_left_right(probability=0.5)
4 p.random_brightness(probability=0.5, min_factor=0.5, max_factor=1.0)
5 p.zoom(probability=0.5, min_factor=0.9, max_factor=1.1)
6 p.random_distortion(probability=0.5, grid_height=10,
7                       grid_width=10, magnitude=10)
8 p.skew(probability=0.5, magnitude=0.12)
9 p.random_erasing(probability=0.3, rectangle_area=0.11)
10 p.rotate(probability=0.5, max_left_rotation=20,
11           max_right_rotation=20)
12 p.sample(4)
```

## 範例：由多張圖片套用多個效果產生多張圖片

建立<image3> 資料夾並上傳<cat.jpg> 及<dog.jpg> 2 個圖片檔，以下程式會以多個圖片處理動作及2 張原始圖片產生4 張圖片。

```
1 import Augmentor
2 p = Augmentor.Pipeline("./image3")
3 p.flip_left_right(probability=0.5)
4 p.random_brightness(probability=0.5, min_factor=0.5, max_factor=1.0)
5 p.zoom(probability=0.5, min_factor=0.9, max_factor=1.1)
6 p.random_distortion(probability=0.5, grid_height=10,
7                       grid_width=10, magnitude=10)
8 p.skew(probability=0.5, magnitude=0.12)
9 p.random_erasing(probability=0.3, rectangle_area=0.11)
10 p.rotate(probability=0.5, max_left_rotation=20,
11           max_right_rotation=20)
12 p.sample(4)
```